

Interview Assignment: Build an AI-Assisted Software Engineering System - URL Shortener

Source: Assignment AI-Proficient Software Engineer1.pdf (Classification: Schwab Internal)

1. Objective

Build a working prototype that transforms a requirement into a reviewable engineering outcome using AI-assisted engineering execution. Demonstrate requirement understanding, task decomposition, multi-step execution, and output generation/validation. Focus on engineer-led execution accelerated by AI, not autonomous orchestration.

2. Scenario

You will build a URL shortener service from scratch with core APIs, analytics, and reliability features. Your task is to complete and improve it over 2-3 days using AI assistance (Copilot/Claude/etc.) while **demonstrating engineering judgment**.

3. Scope

- Greenfield scenarios (new systems/features)
- Brownfield scenarios (enhancements, refactors, bug fixes)
- Test and documentation improvements
- Well-defined and ambiguous requirements

4. Core Requirements

1. Requirement Understanding: Interpret intent, identify ambiguity, normalize into a clear engineering problem.
2. Task Decomposition: Convert high-level requirements into actionable tasks with dependencies and sequencing.
3. Codebase Reasoning (Brownfield): Identify impacted modules/services/APIs/data flows and demonstrate architectural understanding.
4. AI-Assisted Execution (Critical Differentiator): Use AI across implementation, debugging, refactoring, test generation, documentation, and review preparation. Define tasks with intent, constraints, acceptance criteria, and technical context; use disciplined prompting

with iterative refinement; maintain traceability (generated/edited/rejected with rationale); apply quality gates (analysis, linting, tests, security, performance); enforce secure AI usage; require human sign-off for high-impact changes; and retain explicit engineer ownership of correctness, maintainability, and production readiness.

5. Engineering Output Generation: Produce production-quality code, API/schema definitions, unit/integration tests, and supporting documentation with clean design and maintainability.
6. Validation and Risk Control: Identify risks/trade-offs/failure scenarios and define validation and safety guardrails.
7. Controlled Oversight: Engineer leads execution and approves all outputs; AI assists within tasks.
8. Final Engineering Summary: Include plan/rationale, artifacts, risks/trade-offs/validation, assumptions, and limitations.

5. Deliverables

- Working prototype (runnable end-to-end)
- Architecture overview (components, tools, execution approach, control flow, key decisions)
- Three scenarios: greenfield, brownfield, ambiguous (each shows decomposition, execution, validation)
- Setup instructions
- Testing approach, limitations, and trade-offs

6. Evaluation Criteria

- Effectiveness of AI-assisted engineering execution
- Architecture/system design quality
- Depth of decomposition and execution quality
- Realism/quality of outputs
- Validation and risk management rigor
- Clarity and defensibility of decisions
- Core engineering principles: modular, testable, reliable, secure, scalable code with safe change management
- Engineering judgment

7. Expectation

Treat as production-grade engineering work. Demonstrate strong design fundamentals, effective AI use as accelerator, output ownership, and defensible reasoning. Principle: AI assists the engineer within tasks; the engineer owns execution and quality.

Response: :

This whole project was created solely by AI prompts using agents/skills (in .md format)

- 1) Skill to generate a PRD document from a vision document

<https://github.com/dev101-prog/url-shortner/blob/main/docs/Vision%20to%20prd-v1.zip>

- 2) Skill to generate a detail design document from a PRD

<https://github.com/dev101-prog/url-shortner/blob/main/docs/Prd%20to%20detail%20design%20document-v1.zip>

- 3) Agent to generate the code from the requirements and PRD

<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-implementer.md>

Code repository: <https://github.com/dev101-prog/url-shortner/tree/main>

Core Requirements:

- A) Understanding, task module description can be found at the below PRD and design doc

<https://github.com/dev101-prog/url-shortner/blob/main/docs/prd-url-shortener.md>

<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.pdf>

- B) The whole process used AI across defining , implementation , testing, Green/Brown field tasks, traceability, quality gates with a human sign-off at different gates

Deliverables

- Working prototype (runnable end-to-end)
 - <https://github.com/dev101-prog/url-shortner/blob/main/README.md>
- Architecture overview (components, tools, execution approach, control flow, key decisions)
 - <https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.md>
- Three scenarios: greenfield, brownfield, ambiguous (each shows decomposition, execution, validation)
 - See Appendix A below
- Setup instructions
 - <https://github.com/dev101-prog/url-shortner/blob/main/README.md>
- Testing approach, limitations, and trade-offs
 - See section Appendix B below

Appendix A: scenarios

Scenario Coverage: Greenfield, Brownfield and Ambiguous

All three scenario types were covered, following §11 of the design. Each scenario sits on its own branch in the [url-shortner repository](#), with a write-up in [docs/scenarios/](#) on that branch.

Overview: the README's [Scenarios section](#) lists every branch and links to design §11.

Greenfield

Scenario	Branch	Evidence
G1: The new system, all 12 build steps	main	One commit per build step (commit history) and docs/quality-report.md

Brownfield

Scenario	Branch	Write-up
B1: Enhancement: owner-level dedupe default plus the V2 migration	feat/b1-owner-dedupe-default	B1.md
B2: Bug reproduction: the seeded bug plus regression tests that fail	demo/b2-seeded-bug	B2.md
B2: Bug fix: expired link served from cache now returns 410	fix/b2-expired-cache-410	B2.md (includes the root cause)
B3: Refactor: <code>CodeGenerator</code> pulled out	refactor/b3-code-generator	B3.md
B4: Test and documentation improvements	chore/b4-tests-docs	B4.md and the runbook

Ambiguous: "make the analytics more accurate"

All steps are on [feat/a-analytics-accuracy](#).

Step	Write-up
A1: Possible interpretations (I1–I5) with trade-offs	A1.md
A2: The engineer's decision, choosing I1 and I2	ADR 0010
A3: Delivery of <code>exclude_bots</code> (URL-FR-7.9) and unique visitors over the whole range (URL-FR-7.10)	A3.md

Notes

- On every branch except the analytics one, ADR 0010 is only a placeholder that points to `feat/a-analytics-accuracy`.
- How AI was used in each scenario is logged in `docs/ai-usage-log.md` on that scenario's branch.

Appendix B: Testing approach

- **Full strategy:** design §8 Testing Strategy (<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.md#8-testing-strategy>). It covers the test pyramid, test types and tools, the acceptance-test list (each test named after its PRD ID), the ArchUnit rules, the test-data strategy and the k6 performance test.
- **Summary and how to run:** README › Testing strategy (<https://github.com/dev101-prog/url-shortner#testing-strategy>).
- **Measured results:** `docs/quality-report.md` (<https://github.com/dev101-prog/url-shortner/blob/main/docs/quality-report.md>). It has test counts by type, coverage, gate results G1–G12, k6, newman and the demo results.
- **Performance tests:** `docs/perf/README.md` (<https://github.com/dev101-prog/url-shortner/blob/main/docs/perf/README.md>), with the raw k6 results.
- **Quality gates:** design §9 (<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.md#9-quality-gates-and-measurement-of-quality>) and the README (<https://github.com/dev101-prog/url-shortner#quality-gates-and-measurement-of-quality>).

Risks, trade-offs and guardrails

- **Full matrix:** design §12 (<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.md#12-risk-se>

curity-and-operational-mitigations), covering the risk matrix, accepted trade-offs and safety guardrails.

- **Summary:** README › Risks, trade-offs and guardrails
(<https://github.com/dev101-prog/url-shortner#risks-trade-offs-and-guardrails>).
- Each decision with its rationale: docs/adr/
(<https://github.com/dev101-prog/url-shortner/tree/main/docs/adr>), ADRs 0001–0011.
- Failure modes and recovery: docs/runbook.md
(<https://github.com/dev101-prog/url-shortner/blob/main/docs/runbook.md>). The B4 branch extends it with drill commands that were actually run.
- Scaling beyond one instance: docs/scaling.md
(<https://github.com/dev101-prog/url-shortner/blob/main/docs/scaling.md>).

Assumptions and limitations

- Full list: design §16
(<https://github.com/dev101-prog/url-shortner/blob/main/docs/design-url-shortener.md#16-assumptions-limitations-and-deviations-from-the-prd>), covering assumptions, limitations and deviations from the PRD.
- **Summary:** README › Assumptions and limitations
(<https://github.com/dev101-prog/url-shortner#assumptions-and-limitations>).
- Changes from the design made during the build: quality report › design deviations
(<https://github.com/dev101-prog/url-shortner/blob/main/docs/quality-report.md#design-deviations-and-decisions-accepted-so-far>), 19 entries.